

When Does Edge Feedback Help?

A Controlled Study of a Slime-Mould-Inspired Constructor for Ordered Hamiltonian Grid Paths

Author(s) withheld for blinded review

Abstract. Bio-inspired metaheuristics often combine a constructive search policy with a shared adaptive state, making it difficult to identify which component produces observed performance. We study this question for ZipMould, a stochastic constructor for ordered Hamiltonian paths on square grid graphs. Its constraint-aware policy combines waypoint legality, residual-connectivity pruning, onward degree, Manhattan proximity, and checkerboard parity with a nonnegative edge state updated from rank-weighted populations. We compare the complete algorithm with the identical constructor under a single intervention: the edge-state update is frozen at its zero initialization.

To avoid the ceiling found in a legacy 245-puzzle corpus, we introduce Challenge v1, a deterministic, guaranteed-solvable benchmark with planted Hamiltonian certificates, deceptive ordered waypoints, and open, sparse-wall, and chambered instances. Generator strata were selected using only feedback-frozen train/development performance. Before opening a cryptographically committed 150-puzzle test set, we froze the configuration, 30 paired seeds per puzzle, a budget of 3,392 constructed paths per run, the analysis code, and a practical-effect band of ± 5 percentage points.

Full feedback solved 2,039/4,500 runs (45.31%); frozen feedback solved 1,981/4,500 (44.02%). The puzzle-clustered mean difference was +1.29 percentage points, with a predeclared 95% stratified puzzle-bootstrap interval of $[-0.04, +2.64]$ points. Because the complete interval lies inside the practical-effect band, the outcome is classified as practically equivalent. The result does not show that feedback is universally useless; it shows that this implemented edge-feedback package produced no practically important average improvement under the frozen benchmark, configuration, and budget. The study contributes a mechanism-isolating design, a leakage-controlled benchmark, and a reproducible negative result.

Keywords: ordered Hamiltonian path; grid graph; slime mould algorithm; stochastic construction; mechanism ablation; practical equivalence

1. Introduction

Adaptive population metaheuristics are frequently evaluated as indivisible algorithms. A method may combine a strong constructive policy, feasibility filters, tuned domain heuristics, population sampling, and a shared memory update, yet performance is attributed to the bio-inspired update as a whole. This makes a basic causal question surprisingly hard to answer: does the adaptive state improve the probability of solving the target problem, or does the constructor already do the work?

This question matters for slime-mould-inspired optimization. Biological *Physarum* networks adapt their transport structure in response to flow and resource conditions [1]. The continuous Slime Mould Algorithm (SMA) translated related ideas into an adaptive stochastic optimizer with time-varying positive and negative weights [2]. Subsequent work extended SMA to binary feature selection [3], vehicle routing [4], graph problems [5], and hybrids with ant-colony search [6]. Consequently, the scientific value of another discrete adaptation cannot rest on the biological metaphor or on a broad claim of being the first combinatorial SMA. It must identify the implemented mechanism and test what that mechanism adds.

We examine ZipMould, a stochastic constructor for an ordered Hamiltonian grid-path problem. A population of walkers builds self-avoiding paths using hard waypoint and residual-connectivity constraints plus four local heuristics. Between iterations, paths deposit signed rank weights on edges; the resulting edge state decays, receives stochastic edge replacement, and is clipped to a nonnegative range. The central experiment disables that update while holding the constructor and its complete configuration fixed. Because the initial edge state is zero, the comparison asks specifically whether the implemented feedback package changes solve probability.

An earlier 245-puzzle corpus could not answer this question: the tuned full constructor and a same-configuration frozen diagnostic both solved all 1,110 test trials. We therefore treat that corpus as a ceiling diagnostic, not as evidence for feedback. Challenge v1 was built before the confirmatory comparison to obtain nontrivial success rates without selecting instances on a favorable treatment contrast. Its test seed and canonical corpus were cryptographically committed while hidden; code, configuration, paired seed schedule, estimand, bootstrap, and decision rule were frozen at a clean tagged commit before a single acknowledged unlock.

The paper makes three contributions:

1. It gives an implementation-faithful description of an edge-state, rank-weighted stochastic constructor for ordered Hamiltonian grid paths and isolates the complete edge-feedback update with a matched frozen-state intervention.
2. It introduces Challenge v1, a synthetic and redistribution-safe benchmark with planted solution certificates, public train/development splits, a committed held-out test split, and explicit leakage controls.
3. It reports the confirmatory result whether favorable or not. Under 150 puzzles, 30 paired seeds, and 3,392 constructed paths per run, feedback changed average solve probability by +1.29 percentage points (95% interval $[-0.04, +2.64]$), which is practically equivalent under the predeclared ± 5 -point band.

The claim is deliberately narrow. We do not claim state-of-the-art solver performance, formal equivalence for every instance family, or that edge feedback cannot help under other configurations or budgets. The contribution is the controlled evidence about this mechanism in the frozen study.

2. Related work

2.1. Slime-mould optimization and discrete adaptations

SMA was introduced as a continuous stochastic optimizer whose population weights and control parameters change over time [2]. ZipMould is inspired by this adaptive-feedback vocabulary, but it is not a direct transcription: its state belongs to graph edges rather than agents or continuous decision coordinates, and its high-probability random operator replaces individual edge values. This distinction is important because superficial reuse of a metaphor can conceal a materially different algorithm.

Discrete SMA predates this work. Abdel-Basset et al. proposed a binary SMA for feature selection [3]. Zhang et al. applied an improved hybrid SMA to the capacitated vehicle-routing problem [4]. Earlier Physarum-inspired research addressed graph-optimization problems directly [5]. Hybrids also connect Physarum or slime-mould models with ant-colony optimization: Liang et al. used a Physarum-inspired ACO for community mining [7], while Rong et al. combined a slime-mould model, fractional-order memory, and ACO for the travelling-salesman problem [6]. ZipMould’s potential novelty is therefore limited to its precise edge-memory construction and its use for ordered Hamiltonian grid paths, not discrete or graph-based SMA in general.

2.2. Hamiltonian grid paths and constructive memory

Hamiltonian paths in grid graphs are computationally difficult in general; Itai, Papadimitriou, and Szwarcfiter established NP-completeness for relevant grid-graph variants [8]. The ordered-waypoint requirement studied here adds precedence constraints: designated vertices must appear in a fixed sequence along a path that visits every free vertex once. The present work evaluates a stochastic heuristic and does not propose a new complexity result or exact algorithm.

The edge-state sampling rule is also close in spirit to ant-colony optimization (ACO), where construction probabilities depend on shared pheromone and heuristic desirability. The Ant System of Dorigo, Maniezzo, and Colnari is the canonical example [9]. ZipMould differs in its constraint-specific policy, symmetric rank deposits, time-dependent update coefficients, and per-edge replacement noise. We therefore use “slime-mould-inspired” as design provenance, while recognizing ACO as a neighboring algorithmic family.

2.3. Practical equivalence and clustered uncertainty

A non-significant difference does not establish that two methods are meaningfully alike. Equivalence analysis instead requires a smallest effect of interest and evidence that the plausible effect is contained within it [10]. Our frozen rule is not a formal two-one-sided-test procedure. It is a predeclared practical-equivalence classification: a puzzle-cluster bootstrap interval is compared directly with a ± 5 percentage-point band. Bootstrap resampling

follows the general nonparametric principle introduced by Efron [11], with puzzles—not individual seeds—as the independent units.

3. Problem formulation

Let $G = (V, E)$ be the undirected graph induced by the free cells of an $N \times N$ orthogonal grid after wall edges are removed. Let $W = (q_1, \dots, q_K)$ be distinct ordered waypoints. A feasible solution is a vertex sequence

$$P = (v_1, \dots, v_L), \quad L = |V|,$$

such that every vertex of V appears exactly once, consecutive vertices share an edge in E , $v_1 = q_1$, $v_L = q_K$, and the waypoint positions satisfy

$$\text{pos}_{P(q_1)} < \text{pos}_{P(q_2)} < \dots < \text{pos}_{P(q_K)}.$$

Thus, the required path is Hamiltonian over the free cells and respects a total precedence order. Challenge v1 uses no blocked cells, but wall edges alter adjacency. A run is successful only if it returns a path that is revalidated against coverage, uniqueness, adjacency, wall exclusion, endpoints, and waypoint order.

4. ZipMould constructor and feedback

4.1. Constraint-aware path construction

Each iteration initializes n independent walkers at q_1 . A walker maintains its visited set and the index of the most recently reached waypoint. From its current vertex, a neighbor is legal only if it is unvisited, is connected by an open edge, and is not a future waypoint other than the next required one. The constructor additionally rejects a move if deleting the visited prefix and candidate cell disconnects any of the remaining free vertices. This residual-connectivity test is a hard pruning rule, not a soft score.

For a legal candidate edge e , four heuristic terms are computed:

1. **Manhattan proximity** favors geometric proximity to the next waypoint.
2. **Onward degree** uses a Warnsdorff-style preference for candidates with fewer legal onward moves.
3. **Residual connectivity** is zero for candidates that preserve a connected remainder; disconnecting candidates have already been rejected.
4. **Parity balance** weakly favors moves leaving the two checkerboard color classes within one vertex of balance.

After a softplus transform, their product is

$$\eta(e) = \prod_j \text{softplus}(h_{j(e)})^{\gamma_j},$$

where j indexes the four heuristics. With edge state τ_e , the sampling logit is

$$\ell(e) = \alpha\tau_e + \beta \log \eta(e),$$

and the next edge is drawn from the softmax over legal candidates. Unified feedback is used in the confirmatory configuration, so one edge-state field is shared across waypoint segments.

Completed and partial paths are ranked by the fitness

$$f = L_P + \beta_1 k_P + \frac{\beta_2}{1 + d_M} + \beta_3 I_{\text{solved}},$$

where L_P is visited length, k_P is waypoint progress, d_M is Manhattan distance to the next waypoint (the term reaches β_2 after the final waypoint), and the final indicator supplies a large completion bonus. Fitness guides feedback only; the reported endpoint is validated success within a construction budget.

4.2. Rank-weighted edge update

After constructing the population, walkers receive weights by descending fitness rank $r \in \{1, \dots, n\}$:

$$w_r = \frac{n - 2r + 1}{n - 1}.$$

The best path receives +1, the worst −1, and middle ranks interpolate linearly. For edge e , the deposit D_e is the sum of the weights of walkers whose paths traverse e . At zero-indexed iteration t of cap T , the deterministic proposal is

$$\tilde{\tau}_e = v_{c(t)}\tau_e + v_{b(t)}D_e,$$

with $v_{c(t)} = 1 - \frac{t}{T}$ and $v_{b(t)} = \tanh(1 - \frac{t}{T})$. Independently for every edge, with probability z this proposal is replaced by a draw from $\mathcal{N}(0, (\frac{\tau_{\max}}{4})^2)$. The value is then clipped to $[0, \tau_{\max}]$. Lower-ranked paths therefore make subtractive deposits, but the stored state is nonnegative; this should not be described as a positive-only deposit rule.

The feedback-frozen intervention skips the entire update. Since $\tau_0 = 0$, its edge state remains identically zero. All hard constraints, heuristic values, coefficients, path sampling, population size, stopping rules, puzzle, and initial seed remain the same. The intervention isolates the update package—rank deposition, time-dependent memory, and edge replacement noise—but does not identify those subcomponents separately.

Parameter	Value	Parameter	Value
n walkers	53	T iterations	64
Path budget	3,392	α	0.3178
β	0.7347	γ_{man}	0.1218
γ_{warn}	3.0656	γ_{art}	1.3901
γ_{par}	0.3089	z	0.3879
τ_0	0	$\tau_{\max}$	3.1983
Feedback field	unified	State support	nonnegative

Table 1: Frozen Challenge v1 configuration. Both conditions use every value shown; only execution of the edge-state update differs.

5. Challenge v1 benchmark

5.1. Guaranteed-solvable generation

Challenge v1 contains only synthetic content. For each instance, the generator begins with a serpentine Hamiltonian path on the complete $N \times N$ grid and applies $12N^2$ randomized endpoint-backbite moves. These moves alter the certificate while preserving its Hamiltonian property. Ordered waypoints are sampled along the resulting certificate. Interior waypoints preferentially maximize a ratio of certificate distance to Manhattan distance, creating long required segments whose endpoints appear geometrically close.

Walls are then sampled only from edges absent from the certificate, so the planted path always remains feasible. Three generator families control topology: open instances contain no walls; sparse instances independently remove noncertificate edges with probability 0.12; chambered instances add sparse walls and remove most noncertificate edges crossing periodic room boundaries. Certificates are stored separately and used to validate every generated puzzle.

Five family/size strata were selected: chambered 12×12, open 14×14, open 16×16, sparse 12×12, and sparse 14×14. Each stratum contributes 40 public training puzzles, 20 public development puzzles, and 30 held-out test puzzles, for 200/100/150 puzzles. Strata were retained only when feedback-frozen development success was informative. The calibration used five seeds per development puzzle and produced an overall frozen solve rate of 0.458; all selected stratum rates lay within the prespecified 0.15–0.85 target range. The full-minus-frozen contrast was never used to select generator parameters, strata, configuration, or budget.

5.2. Test commitment and release discipline

The test split was generated from a random 256-bit seed kept outside tracked artifacts. Before test access, the repository committed a domain-separated seed hash, canonical-corpus hash, certificate hash, exact stratum counts, and public specification hash. Verification regenerated and hashed the test corpus in memory without writing puzzle contents. Final execution required a clean commit carrying the annotated tag `challenge-v1-confirmatory-v1`.

After code, configuration, schedule, and analysis were frozen, the test was unlocked once with an explicit final-analysis acknowledgement. The complete corpus, seed reveal, certificates, raw trials, manifest, and derived reports were archived after analysis. This is a repository-level cryptographic commitment rather than registration with an external preregistration service.

6. Confirmatory design and analysis

6.1. Conditions, pairing, and budget

The two conditions were full feedback and feedback frozen. Each used the same configuration in Table 1. For each of 150 puzzles, both conditions were run with seeds 0–29, producing 9,000 required rows. Population 53 and iteration cap 64 give a maximum of 3,392 constructed paths per run; this was the scientific budget. A 300-second wall-clock guard was retained only as an engineering failsafe and was never reached.

Seed pairing means that conditions begin from the same derived kernel seed for a puzzle. It does not create common random numbers after the intervention: feedback updates consume random draws in the full arm but are skipped in the frozen arm, so later random streams diverge. Pairing is preserved in descriptive outcome counts, while puzzle remains the independent analysis unit.

6.2. Estimand, interval, and decision rule

For puzzle p , let $s_{F(p)}$ and $s_0(p)$ be the proportions of its 30 seeds solved under full and frozen feedback. The primary estimand is

$$\Delta = \frac{1}{150} \sum_{p=1}^{150} (s_{F(p)} - s_0(p)).$$

Uncertainty was computed with 10,000 stratified cluster-bootstrap replicates. Within each of the five frozen strata, 30 puzzles were sampled with replacement; the paired per-puzzle condition difference was retained, and the 150 sampled effects were averaged. The bootstrap used a local SplitMix64 stream with seed 20260716. The 95% endpoints were the prespecified empirical order statistics at indices $\lfloor 0.025B \rfloor$ and $\lceil 0.975B \rceil - 1$ for $B = 10,000$.

Before test unlock, ± 5 percentage points was declared the smallest practically important average effect. The outcome labels were:

1. **helpful** if the full interval was above +0.05;
2. **harmful** if the full interval was below -0.05 ;
3. **practically equivalent** if the full interval was inside $[-0.05, +0.05]$; and
4. **inconclusive** otherwise.

This rule differs from declaring equivalence after a non-significant test. It also differs from a formal TOST analysis; it is an interval-and-band classification frozen for this study. Marginal condition rates, five stratum effects, per-puzzle effect directions, and paired seed outcomes were predeclared as descriptive secondary results. Iteration counts and wall time were archived but were not confirmatory efficiency endpoints.

6.3. Integrity checks

Analysis required the exact $150 \times 30 \times 2$ grid, without duplicate or failed rows. Every row had to match the frozen configuration hash, tag commit, seed schedule, budget, and intervention field. All 4,020 returned solutions were independently revalidated from their archived coordinate paths. After the first analysis, a second run in a separate directory reproduced the report, puzzle-effect table, and all bootstrap replicates byte for byte.

7. Results

Full feedback solved 2,039 of 4,500 trials (45.31%), compared with 1,981 of 4,500 (44.02%) for frozen feedback (Table 2). The primary mean full-minus-frozen difference was +0.01289, or +1.29 percentage points. Its 95% stratified puzzle-bootstrap interval was $[-0.00044, +0.02644]$, equivalent to $[-0.04, +2.64]$ percentage points (Figure 1). The entire interval lies inside the predeclared ± 5 -point practical-effect band; the confirmatory classification is therefore **practically equivalent**.

Condition	Solved	Trials	Solve rate
Full feedback	2,039	4,500	0.453
Feedback frozen	1,981	4,500	0.440

Table 2: Confirmatory marginal success counts. The primary analysis uses clustered per-puzzle differences rather than treating 4,500 trials as independent.

Effect of edge feedback on solve probability

Overall interval is confirmatory; stratum estimates are descriptive.

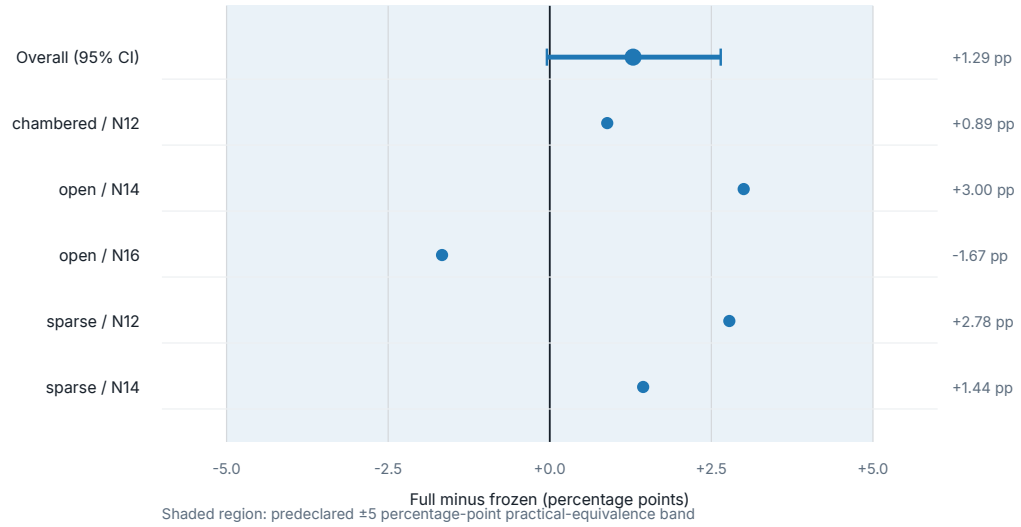


Figure 1: Full-minus-frozen solve-probability effects. The overall interval is the prespecified stratified puzzle-bootstrap interval. Stratum intervals are not inferential claims; stratum point estimates are descriptive. The shaded band marks ± 5 percentage points.

The five descriptive stratum effects ranged from -1.67 points for open 16×16 to $+3.00$ points for open 14×14 (Table 3). No stratum point estimate crossed either practical boundary. These estimates were not separately powered or multiplicity-adjusted and should not be read as evidence of family-specific benefit.

Stratum	Full	Frozen	Difference
Chambered, 12×12	0.379	0.370	+0.009
Open, 14×14	0.684	0.654	+0.030
Open, 16×16	0.279	0.296	-0.017
Sparse, 12×12	0.753	0.726	+0.028
Sparse, 14×14	0.170	0.156	+0.014

Table 3: Predeclared descriptive results by generator stratum. Each row contains 30 puzzles, 30 seeds per puzzle, and 900 trials per condition.

Across the 4,500 paired puzzle-seed combinations, both arms solved 1,574, only full feedback solved 465, only frozen feedback solved 407, and neither solved 2,054. At puzzle level, full feedback had a higher seed solve rate on 57 puzzles, the arms tied on 48, and frozen feedback was higher on 45. The distribution of per-puzzle effects is heterogeneous but centered near zero (Figure 2); the secondary displays do not alter the primary classification.

Per-puzzle effects across Challenge v1

Each point is one puzzle's difference over 30 paired seeds.

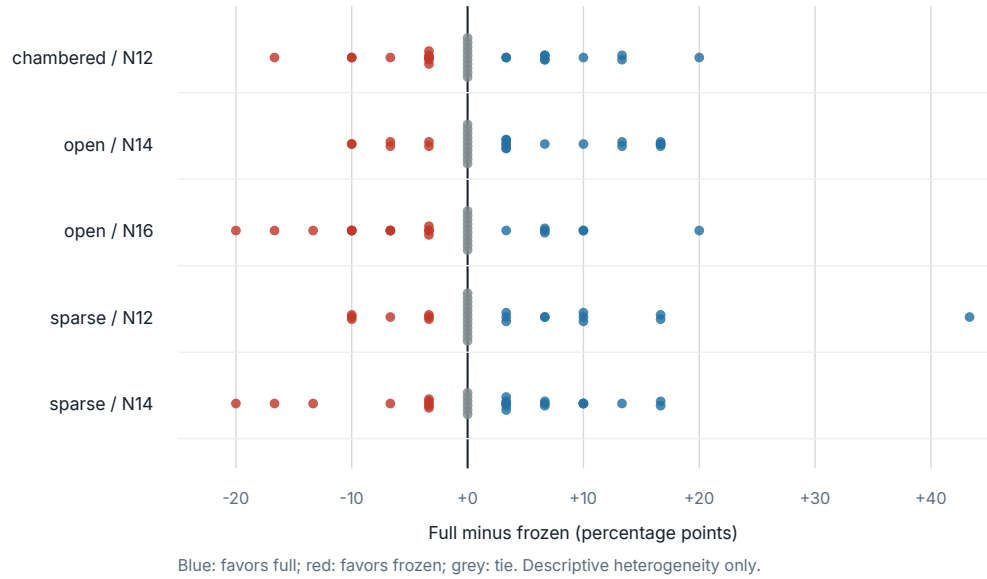


Figure 2: All 150 per-puzzle differences, sorted within stratum. Blue favors full feedback, red favors frozen feedback, and gray is a tie. This plot is descriptive.

8. Discussion

The controlled result is negative in the scientifically useful sense: the implemented edge-feedback package did not produce a practically important average improvement under the frozen Challenge v1 design. The point estimate favors feedback slightly, but the full 95% interval—from a negligible disadvantage to a 2.64-point advantage—remains well inside the ± 5 -point band. Describing this result only as “not significant” would miss the predeclared practical claim; describing it as universal equivalence would overstate it.

The result clarifies the legacy ceiling. On the original test corpus, the tuned full constructor solved 1,110/1,110 trials, and a post-hoc same-configuration frozen diagnostic also solved 1,110/1,110. Challenge v1 lowered both methods into an informative success regime, yet the average contrast remained small. Together, these observations suggest that the constraint-aware constructor—not its adaptive edge state—is the dominant source of performance in the studied settings.

Several mechanisms could explain the small contrast. First, hard residual-connectivity pruning is powerful: it removes moves that would make Hamiltonian completion impossible. Second, the onward-degree heuristic directly encodes a classic constructive principle, while waypoint legality and parity balance further narrow the choice set. Third, the relatively high edge-replacement probability ($z = 0.3879$ independently per edge per update) can disrupt accumulated state as well as diversify it. Finally, 64 iterations may be sufficient for the constructor to sample many useful paths but too short, too long, or otherwise mismatched for stable edge learning. The present intervention intentionally does not decide among these explanations.

Methodologically, the study illustrates why mechanism ablations should preserve the surrounding algorithm. A separate “heuristic baseline” with a different population or budget cannot identify the effect of feedback. Freezing a zero-initialized state inside the same implementation removes a much narrower component. Likewise, a hard benchmark selected because the proposed method wins can bias the treatment contrast. Challenge v1 used only frozen-arm difficulty for calibration and concealed the confirmatory split until the analysis was immutable.

The practical-equivalence band is a scientific judgment rather than a fact supplied by the data. Five percentage points was chosen as the minimum improvement that would justify the added state update for the study’s primary endpoint. Readers who prefer a narrower threshold can still use the reported interval: it rules out average improvements above

2.64 points at the chosen confidence level but does not establish equivalence inside, for example, ± 1 point. Reporting the estimate and interval keeps that sensitivity visible.

The outcome does not imply that shared edge memory has no research value. Heterogeneous effects and the mechanistic structure motivate new, explicitly exploratory work: separately remove rank-negative deposits, decay, and edge replacement; vary the construction budget; learn state per waypoint segment; or identify instance features that modify the effect. Such studies should use new development data and a newly locked confirmatory split. They should not repurpose the released Challenge v1 test set for another confirmatory claim.

9. Limitations and threats to validity

Synthetic instances. Challenge v1 guarantees solvability and avoids third-party data rights, but its instances arise from one planted-certificate generator. Backbite-randomized paths, deceptive waypoint placement, and certificate-preserving walls may not represent human-authored puzzles or other Hamiltonian graph distributions.

Limited domain and scales. The confirmatory set covers five family/size strata and square grids of size 12, 14, or 16. Conclusions may not generalize to larger grids, blocked cells, non-grid graphs, other waypoint densities, or different wall processes.

One configuration and budget. The experiment estimates the effect of feedback for one dev-selected configuration under 3,392 constructed paths. It does not average over hyperparameters or show that either arm is optimally tuned. Interactions between feedback and population, iteration cap, state clipping, or noise may change the effect.

Composite intervention. Freezing the update simultaneously removes signed rank deposits, temporal decay, and stochastic edge replacement. The design has strong isolation at the package level but cannot attribute the result to a subcomponent. The nonnegative state also clips negative aggregate deposits; calling it “positive-only” would be inaccurate.

Random-stream divergence. Conditions share their initial derived seed, but feedback consumes random draws, so their streams cease to be synchronized. The comparison is between two stochastic algorithms with paired initialization, not a common-random-number estimate at every construction decision.

Practical band and interval. The ± 5 -point band was predeclared but remains domain judgment. The empirical percentile cluster bootstrap is transparent and reproducible, but other interval procedures could differ in finite samples. The classification is not a formal TOST result.

No state-of-the-art comparison. The mechanism question requires a matched constructor ablation, not a leaderboard. This paper therefore does not establish superiority over exact solvers, constraint programming, specialized ACO, or other tuned metaheuristics. Archived legacy baselines are contextual only and are not used for the confirmatory claim.

10. Reproducibility and data availability

The software, benchmark specification, public train/development corpus, test commitment, frozen protocol, and analysis are included with the artifact. The pre-test release is Git commit `06ef8bbfae5de29fe6cd4ebf0175a94f04880e9b` and annotated tag `challenge-v1-confirmatory-v1`. The post-analysis evidence directory contains all 9,000 raw rows, 4,020 solved coordinate paths, the exact test corpus and certificates, the revealed seed, 10,000 bootstrap replicates, per-puzzle effects, environment and hardware metadata, and SHA-256 checksums.

The raw result SHA-256 is `a899646ef363ad2fd295e704b35f73b22efeb08ad10ebd5dc6429e1d32b1976a`. Execution used Python 3.13.12, NumPy 2.4.4, Numba 0.65.1, and Polars 1.40.1 on an AMD Ryzen 9 9950X3D system with 32 logical CPUs. The complete 9,000-run grid finished in 145.0 seconds with no failed rows; timing is provenance, not a comparative endpoint.

From the repository root, `make paper-verify` checks the frozen protocol, `make paper-smoke` exercises both arms on public data, `make paper-analyze` regenerates the analysis, and `make paper-figures` regenerates publication displays. The evidence package documents seed reconstruction and byte-level verification. Author-identifying repository and archive URLs are omitted from this blinded manuscript and must be inserted in the camera-ready data-availability statement.

11. Conclusion

ZipMould’s edge-state feedback was tested against the same stochastic constructor with the update frozen, under a sealed and reproducible Challenge v1 protocol. Full feedback improved average solve probability by 1.29 percentage points, with a 95% puzzle-bootstrap interval from -0.04 to $+2.64$ points. This interval lies within the predeclared ± 5 -point band, yielding a practically equivalent classification. The evidence therefore supports a narrow conclusion: for this implementation, benchmark, configuration, and budget, the constraint-aware constructor performed essentially as well without adaptive edge feedback. Publishing that result—together with the test commitment, raw paths, and exact analysis—provides a firmer basis for future mechanism design than attributing ceiling performance to a biological metaphor.

12. Declarations

Ethics approval: Not applicable; the study uses synthetic benchmark instances and no human or animal participants.

Author contributions, funding, competing interests, and acknowledgements: Withheld or to be completed by the authors before non-blinded submission.

References

- [1] A. Tero *et al.*, “Rules for Biologically Inspired Adaptive Network Design,” *Science*, vol. 327, no. 5964, pp. 439–442, 2010, doi: [10.1126/science.1177894](https://doi.org/10.1126/science.1177894).
- [2] S. Li, H. Chen, M. Wang, A. A. Heidari, and S. Mirjalili, “Slime mould algorithm: A new method for stochastic optimization,” *Future Generation Computer Systems*, vol. 111, pp. 300–323, 2020, doi: [10.1016/j.future.2020.03.055](https://doi.org/10.1016/j.future.2020.03.055).
- [3] M. Abdel-Basset, R. Mohamed, R. K. Chakraborty, M. J. Ryan, and S. Mirjalili, “An efficient binary slime mould algorithm integrated with a novel attacking-feeding strategy for feature selection,” *Computers & Industrial Engineering*, vol. 153, p. 107078, 2021, doi: [10.1016/j.cie.2020.107078](https://doi.org/10.1016/j.cie.2020.107078).
- [4] X. Zhang, Q. Liu, and X. Bai, “Improved slime mould algorithm based on hybrid strategy optimization of Cauchy mutation and simulated annealing,” *PLOS ONE*, vol. 18, no. 1, p. e280512, 2023, doi: [10.1371/journal.pone.0280512](https://doi.org/10.1371/journal.pone.0280512).
- [5] X. Zhang, C. Gao, Y. Deng, and Z. Zhang, “Slime Mould Inspired Applications on Graph-Optimization Problems,” *Advances in Physarum Machines: Sensing and Computing with Slime Mould*, vol. 21. in Emergence, Complexity and Computation, vol. 21. Springer, Cham, pp. 519–562, 2016. doi: [10.1007/978-3-319-26662-6_26](https://doi.org/10.1007/978-3-319-26662-6_26).
- [6] Z. Rong, X. Gong, X. Wang, W. Lv, and J. Wang, “A Slime Mold Fractional-Order Ant Colony Optimization Algorithm for Travelling Salesman Problems,” in *Advances in Swarm Intelligence*, in Lecture Notes in Computer Science, vol. 12689. Cham: Springer, 2021, pp. 322–332. doi: [10.1007/978-3-030-78743-1_29](https://doi.org/10.1007/978-3-030-78743-1_29).
- [7] M. Liang, C. Gao, X. Li, and Z. Zhang, “A Physarum-Inspired Ant Colony Optimization for Community Mining,” in *Advances in Knowledge Discovery and Data Mining*, in Lecture Notes in Computer Science, vol. 10234. Cham: Springer, 2017, pp. 737–749. doi: [10.1007/978-3-319-57454-7_57](https://doi.org/10.1007/978-3-319-57454-7_57).
- [8] A. Itai, C. H. Papadimitriou, and J. L. Szwarcfiter, “Hamilton Paths in Grid Graphs,” *SIAM Journal on Computing*, vol. 11, no. 4, pp. 676–686, 1982, doi: [10.1137/0211056](https://doi.org/10.1137/0211056).
- [9] M. Dorigo, V. Maniezzo, and A. Colnari, “Ant system: Optimization by a colony of cooperating agents,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 26, no. 1, pp. 29–41, 1996, doi: [10.1109/3477.484436](https://doi.org/10.1109/3477.484436).
- [10] D. Lakens, “Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses,” *Social Psychological and Personality Science*, vol. 8, no. 4, pp. 355–362, 2017, doi: [10.1177/1948550617697177](https://doi.org/10.1177/1948550617697177).
- [11] B. Efron, “Bootstrap Methods: Another Look at the Jackknife,” *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: [10.1214/aos/1176344552](https://doi.org/10.1214/aos/1176344552).